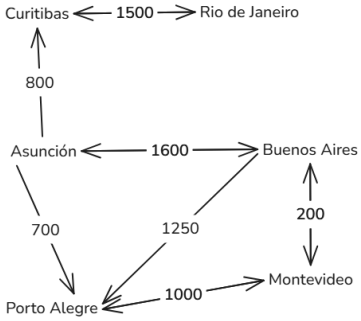
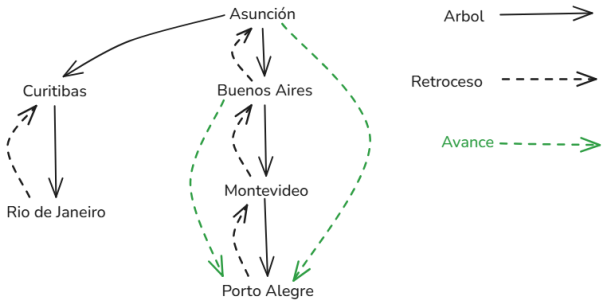
A la izquierda, el grafo dirigido; a la derecha, el bosque abarcador del DFS desde Asunción con los arcos de árbol, retroceso y avance.

Asunción → (Buenos Aires, 1600), (Curitiba, 800), (Porto Alegre, 700)
Buenos Aires → (Asunción, 1600), (Montevideo, 200), (Porto Alegre, 1250)
Curitiba → (Rio de Janeiro, 1500)
Montevideo → (Buenos Aires, 200), (Porto Alegre, 1000)
Porto Alegre → (Montevideo, 1000)
Rio de Janeiro → (Curitiba, 1500)





Los adyacentes se visitan en el orden de la tabla. Orden de visita: **Asunción, Buenos Aires, Montevideo, Porto Alegre, Curitiba, Rio de Janeiro**.

Asunción: visita Buenos Aires.
Buenos Aires: Asunción ya está en la pila (retroceso); visita Montevideo.
Montevideo: Buenos Aires en la pila (retroceso); visita Porto Alegre.
Porto Alegre: Montevideo en la pila (retroceso). Termina y vuelve.
De vuelta en Buenos Aires: Porto Alegre ya fue visitado y es su descendiente (avance).
De vuelta en Asunción: visita Curitiba, y desde ella Rio de Janeiro.
Rio de Janeiro: Curitiba en la pila (retroceso).
De vuelta en Asunción: Porto Alegre ya visitado y descendiente (avance).

Tipo de arco	**Arcos**
Árbol (5)	Asunción→Buenos Aires, Buenos Aires→Montevideo, Montevideo→Porto Alegre, Asunción→Curitiba, Curitiba→Rio de Janeiro
Retroceso (4)	Buenos Aires→Asunción, Montevideo→Buenos Aires, Porto Alegre→Montevideo, Rio de Janeiro→Curitiba

Avance (2)	Buenos Aires→Porto Alegre, Asunción→Porto Alegre
Cruzado (0)	–

El método está declarado en IDirectedGraphAlgorithms y se implementa en DirectedGraphAlgorithms. Recibe el grafo, el vértice de inicio y un Consumer que procesa cada vértice visitado. Como sugiere la pista, un método auxiliar recursivo recibe el conjunto de visitados.

```
@Override
public <V, D> void recorridoEnProfundidad(IGraph<V, D> grafo, Comparable<V> sourceCriteria,
                                          Consumer<V> consumer) {
  V origen = grafo.buscarVertice(sourceCriteria);
  if (origen == null) {
    return;
  }
  dfs(grafo, origen, new HashSet<>(), consumer);
}

private <V, D> void dfs(IGraph<V, D> grafo, V actual, Set<V> visitados, Consumer<V> consumer) {
  visitados.add(actual);
  consumer.accept(actual);
  for (Edge<V, D> arista : grafo.adyacencias(grafo.construirComparable(actual))) {
    V destino = arista.target();
    if (!visitados.contains(destino)) {
      dfs(grafo, destino, visitados, consumer);
```

```
    }
  }
}
```

Orden: O(V + E) en la recursión; cada llamada a adyacencias además busca el vértice en O(V).
Si el vértice de inicio no existe, no se visita nada.
El método default sin vértice de inicio llama a este una vez por cada vértice, cada vez con visitados nuevos, por lo que puede repetir vértices. Para recorrer el bosque completo hay que compartir el conjunto de visitados entre las llamadas.

Caso de prueba: con el grafo cargado en el orden de la tabla y sourceCriteria = Asunción, el Consumer recibe Asunción, Buenos Aires, Montevideo, Porto Alegre, Curitiba, Rio de Janeiro.